

COMPARISON · SPLIT · STRUCTURE

compare-code

Two fenced code blocks side-by-side, each with a label.

BEFORE & AFTER · COMPONENT MANIFEST LOADING

Flat-file lookup versus folder-shape lookup.

BEFORE · FLAT FILE

```
const fs = require('node:fs');
const path = require('node:path');

function loadOne(name) {
  const p = path.join(
    __dirname, 'lib', 'components',
    `${name}.json`
  );
  return JSON.parse(fs.readFileSync(p, 'utf8'));
}

const cards = loadOne('cards-grid');
```

AFTER · FOLDER SHAPE

```
const fs = require('node:fs');
const path = require('node:path');

function loadOne(name) {
  const p = path.join(
    __dirname, 'lib', 'components',
    name, 'manifest.json'
  );
  return JSON.parse(fs.readFileSync(p, 'utf8'));
}

const cards = loadOne('cards-grid');
```

AFTER & BEFORE · COMPONENT MANIFEST LOADING

Folder-shape lookup, with the prior approach for reference.

AFTER · FOLDER SHAPE

```
function loadOne(name) {  
  const p = path.join(  
    __dirname, 'lib', 'components',  
    name, 'manifest.json'  
  );  
  return JSON.parse(fs.readFileSync(p, 'utf8'));  
}
```

BEFORE · FLAT FILE

```
function loadOne(name) {  
  const p = path.join(  
    __dirname, 'lib', 'components',  
    `${name}.json`  
  );  
  return JSON.parse(fs.readFileSync(p, 'utf8'));  
}
```

BEFORE & AFTER · COMPONENT MANIFEST LOADING

Flat-file lookup versus folder-shape lookup.

BEFORE · FLAT FILE

```
const fs = require('node:fs');
const path = require('node:path');

function loadOne(name) {
  const p = path.join(
    __dirname, 'lib', 'components',
    `${name}.json`
  );
  return JSON.parse(fs.readFileSync(p, 'utf8'));
}

const cards = loadOne('cards-grid');
```

AFTER · FOLDER SHAPE

```
const fs = require('node:fs');
const path = require('node:path');

function loadOne(name) {
  const p = path.join(
    __dirname, 'lib', 'components',
    name, 'manifest.json'
  );
  return JSON.parse(fs.readFileSync(p, 'utf8'));
}

const cards = loadOne('cards-grid');
```

BEFORE & AFTER · COMPONENT MANIFEST LOADING

Flat-file lookup versus folder-shape lookup.

BEFORE · FLAT FILE

```
const fs = require('node:fs');
const path = require('node:path');

function loadOne(name) {
  const p = path.join(
    __dirname, 'lib', 'components',
    `${name}.json`
  );
  return JSON.parse(fs.readFileSync(p, 'utf8'));
}

const cards = loadOne('cards-grid');
```

AFTER · FOLDER SHAPE

```
const fs = require('node:fs');
const path = require('node:path');

function loadOne(name) {
  const p = path.join(
    __dirname, 'lib', 'components',
    name, 'manifest.json'
  );
  return JSON.parse(fs.readFileSync(p, 'utf8'));
}

const cards = loadOne('cards-grid');
```

BEFORE & AFTER · COMPONENT MANIFEST LOADING

Flat-file lookup versus folder-shape lookup.

BEFORE · FLAT FILE

```
const fs = require('node:fs');
const path = require('node:path');

function loadOne(name) {
  const p = path.join(
    __dirname, 'lib', 'components',
    `${name}.json`
  );
  return JSON.parse(fs.readFileSync(p, 'utf8'));
}

const cards = loadOne('cards-grid');
```

AFTER · FOLDER SHAPE

```
const fs = require('node:fs');
const path = require('node:path');

function loadOne(name) {
  const p = path.join(
    __dirname, 'lib', 'components',
    name, 'manifest.json'
  );
  return JSON.parse(fs.readFileSync(p, 'utf8'));
}

const cards = loadOne('cards-grid');
```

When NOT to reach for compare-code.

One side is prose. If one column is code and the other is description, use a single fenced block with surrounding prose. `compare-code` is for code-versus-code.

Snippets longer than 14 lines. The text shrinks below readability past 14 lines per side. Split into two slides or extract the key delta into a smaller diff.

Three-way comparison. `compare-code` is binary. For three configurations or three implementations, use prose with successive fenced blocks or a `compare-table`.

See also.

RELATED COMPONENTS

- `before-after` — the change is state, not code
- `compare-prose` — the comparison is prose-versus-prose
- `redline` — the change is in verbatim text or legal language
- `compare-table` — three or more variants on shared dimensions